Python Basics · Day 2 of 30 · about 60 minutes

Variables and data types

🎯 Today's goal

By the end of this hour you will:

- store values in **variables** and use them again later
- update a variable, for example adding points to a score
- know the four basic **data types**: whole numbers, decimal numbers, text and True/False
- convert between types, for example the text `"42"` into the number `42`
- build neat messages with **f-strings**

🤔 Why this matters

Yesterday every number disappeared as soon as Python printed it. Real data work needs Python to **remember** things: a price, a temperature, a customer's name.

A variable is like a **labelled box**: you put a value inside and write a name on the label. Later you ask for the box by its name. In a spreadsheet, the column names (*age*, *city*, *salary*) play the same role — and data types decide whether a column holds numbers you can add up or text you can't.

📦 Creating a variable

You create a variable with a **name**, an equals sign and a **value**. Then you can use the name anywhere instead of the value:

```
city = "Cairo"
temperature = 31

print(city)
print(temperature)
print("Today in", city, "it is", temperature, "degrees")
```

```
Cairo
31
Today in Cairo it is 31 degrees
```

Careful: in Python, `=` does not mean "is equal to". It means **"store the value on the right in the name on the left"**. Read `temperature = 31` as "temperature *becomes* 31".

Variables can be used in calculations, and the result can go into a new variable:

```
price = 12.5
quantity = 4
total = price * quantity

total
```

```
50.0
```

Try changing `quantity` to `10` and run the box again. The total updates — that's the power of variables: change one value, and everything built on it follows.

🔄 Changing a variable

A variable can be given a new value at any time. The old value is thrown away. Very often the new value is *based on* the old one, like adding points to a score:

```
score = 10
print("Start:", score)

score = score + 5     # take the old score, add 5, store it back
print("After a bonus:", score)

score += 5            # a shortcut that does exactly the same
print("After another bonus:", score)
```

```
Start: 10
After a bonus: 15
After another bonus: 20
```

There's a shortcut for each maths operator:

Shortcut	Means the same as	In words
`x += 3`	`x = x + 3`	add 3 to x
`x -= 3`	`x = x - 3`	take 3 away from x
`x *= 3`	`x = x * 3`	multiply x by 3
`x /= 3`	`x = x / 3`	divide x by 3

```
savings = 100
savings -= 30     # bought a jacket
savings *= 2      # a generous gift doubled the rest
savings
```

```
140
```

🏷️ Naming your variables

Python has a few rules for names:

- Use letters, numbers and the underscore `_`. No spaces.
- A name can't start with a number: `2nd_place` is not allowed, `second_place` is.
- Capital letters matter: `Age` and `age` are two different variables.

Python programmers write names in **snake_case**: all lowercase, with underscores between words, like `monthly_salary` or `average_temperature`. Choose names that say what's inside — `total_price` is much clearer than `tp` or `x`.

Using a name that doesn't exist (often a typo) gives a `NameError`:

💡 This example has a mistake on purpose. Run it and read the error.

```
average_temperature = 24.5
print(average_temprature)
```

> ⚠️ **NameError**
>
> Python doesn't know this name. Check the spelling (capital letters matter), put quotes around it if it's text, or run the box that creates it first.
>
> ```
> Traceback (most recent call last):
>   line 2, in <module>
>     print(average_temprature)
>           ^^^^^^^^^^^^^^^^^^
> NameError: name 'average_temprature' is not defined. Did you mean:
> 'average_temperature'?
> ```

Can you spot the typo? (`temprature` is missing an "e".)

🧱 Data types

Every value in Python has a **type**: the kind of thing it is. The type decides what you can do with it — you can multiply numbers, but you can't multiply two names. These are the four basic types:

Type	Short for	What it holds	Examples
`int`	integer	whole numbers	`31`, `-4`, `2026`
`float`	floating-point number	numbers with a decimal point	`12.5`, `0.07`, `-3.0`
`str`	string	text, always in quotes	`"Cairo"`, `'42'`
`bool`	boolean	only `True` or `False`	`True`, `False`

The `type()` function tells you the type of any value or variable:

```
print(type(31))
print(type(12.5))
print(type("Cairo"))
print(type(True))

print(type(city))       # variables have the type of the value inside them
print(type(total))
```

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'str'>
<class 'float'>
```

Notice that `total` is a `float`: `12.5 * 4` gives `50.0`, because one of the numbers had a decimal point.

Booleans (`True` / `False`, always with a capital letter) answer yes/no questions. You can get one by comparing values. We'll use them a lot on Day 4:

```
is_raining = False
print("Is it raining?", is_raining)

is_hot = temperature > 30      # is 31 greater than 30?
print("Is it hot?", is_hot)
```

```
Is it raining? False
Is it hot? True
```

🔁 Converting between types

Data often arrives as **text**, even when it looks like a number — for example when it's read from a file or a web form. Before you can do maths with it, you convert it. Each type has a function with the same name that does this:

```
age_text = "42"
print(age_text + "8")        # text: glued together
print(int(age_text) + 8)     # converted to a whole number: added

print(float("3.5") * 2)      # text to decimal number
print(str(25) + " years")    # number to text, so it can be glued to text
```

```
428
50
7.0
25 years
```

Converting a decimal number to `int` **chops off** the decimals — it doesn't round. To round properly, use `round()`, optionally with the number of decimals you want:

```
print(int(3.99))          # chops: 3
print(round(3.99))        # rounds: 4
print(round(3.14159, 2))  # rounds to 2 decimals: 3.14
```

```
3
4
3.14
```

Python can only convert text that actually looks like a number. Otherwise you get a `ValueError`:

💡 This example has a mistake on purpose. Run it and read the error.

```
int("hello")
```

> ⚠️ **ValueError**
>
> The kind of value is right but the value itself isn't, like int("hello").
>
> ```
> Traceback (most recent call last):
>   line 1, in <module>
>     int("hello")
>     ~~~^^^^^^^^^
> ValueError: invalid literal for int() with base 10: 'hello'
> ```

✨ Neat messages with f-strings

Gluing text with `+` and `str()` gets messy fast. An **f-string** is easier: put an `f` before the opening quote, and write variables inside curly brackets `{ }`. Python fills in their values:

```
name = "Sara"
age = 21

print(f"{name} is {age} years old.")
print(f"Next year {name} will be {age + 1}.")   # you can even do maths inside { }
```

```
Sara is 21 years old.
Next year Sara will be 22.
```

You can also control how numbers look. Add a colon and a format after the name: `:.2f` means "show 2 decimals", `:.1f` means 1 decimal.

```
average = 58.6 / 3
print(average)                                  # many decimals
print(f"Average: {average:.1f} degrees")        # 1 decimal
print(f"Price: {9.5:.2f} dollars")              # 2 decimals, like money
```

```
19.533333333333335
Average: 19.5 degrees
Price: 9.50 dollars
```

🧰 A few handy text tools

Strings come with built-in tools called **methods**. You use a method by writing a dot after the value, then the method's name and brackets. These are great for tidying up messy data:

```
messy_city = "   cAIRo  "

print(len(messy_city))           # len() counts characters, spaces included
clean_city = messy_city.strip()  # strip() removes spaces at the start and end
print(clean_city)
print(clean_city.lower())        # all lowercase
print(clean_city.upper())        # ALL UPPERCASE
print(clean_city.title())        # First Letter Capital
print("data-science".replace("-", " "))
```

```
10
cAIRo
cairo
CAIRO
Cairo
data science
```

⚠️ Common mistakes

- Reading `=` as "equals". It means "store in". (Comparing uses `==`, coming on Day 4.)
- Using a variable before creating it, or with a typo → `NameError`.
- Putting quotes around a variable name: `print("city")` prints the word *city*, `print(city)` prints what's inside.
- Doing maths with numbers stored as text: `"5" + 3` → convert first with `int()` or `float()`.
- Forgetting the `f` in an f-string: `"{name}"` prints the curly brackets literally.

✅ Recap

- `name = value` stores a value in a variable; use the name to get it back.
- `+=`, `-=`, `*=`, `/=` update a variable based on its old value.
- Names: lowercase snake_case, no spaces, can't start with a number.
- Four basic types: `int`, `float`, `str`, `bool`. Check with `type()`.
- Convert with `int()`, `float()`, `str()`; round with `round()`.
- f-strings: `f"{name} is {age}"`, and `{value:.2f}` for 2 decimals.

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Solutions are in the answer key at the end.

1. Your profile

Create three variables:

- `name` : your name (text)
- `age` : your age as a whole number
- `height` : your height in metres, with a decimal point (like `1.72`)

```
# Create name, age and height below
```

Hints:

1. Text needs quotes, numbers don't: `name = "Sara"`
2. A whole number has no decimal point (`age = 21`); a height needs one (`height = 1.65`).

2. Game score

A player starts with `score = 10`. Using the shortcuts from today, **add 5** points with `+=`, then **double** the score with `*=`. Don't just type the final number!

```
score = 10

# Add 5 points

# Double the score
```

Hints:

1. `score += 5` adds 5 to the score.
2. `score *= 2` doubles it. The final score should be 30.

3. Numbers that arrived as text

An online shop sent its data as text. Convert the values and calculate the **total cost** (price × quantity) in a variable called `total`.

```
price_text = "19.99"
quantity_text = "3"

# Convert and calculate total
```

Hints:

1. The price has decimals, so use `float(price_text)`.
2. The quantity is a whole number: `int(quantity_text)`.
3. `total = float(price_text) * int(quantity_text)`

4. Average temperature

Calculate the average of the three temperatures in a variable called `average_temp`, then print it with an f-string and **1 decimal**, like this: `Average: 19.5 degrees`

```
monday = 17.2
tuesday = 21.9
wednesday = 19.4

# Calculate average_temp, then print it
```

Hints:

1. Average = add them up, then divide by how many there are. Use brackets: `(monday + tuesday + wednesday) / 3`
2. 1 decimal inside an f-string: `{average_temp:.1f}`
3. `print(f"Average: {average_temp:.1f} degrees")`

5. Challenge: test report

A student answered some questions correctly out of a total. Calculate the **percentage** in a variable called `percentage` (correct ÷ total × 100), then print a report using the variables, like: `Lina scored 85.0%`

Your code should still work if someone changes the name or the numbers at the top.

```
student = "Lina"
correct = 17
total_questions = 20

# Calculate percentage, then print the report
```

Hints:

1. `percentage = correct / total_questions * 100`
2. Use the variables inside an f-string: `{student}` and `{percentage:.1f}`
3. `print(f"{student} scored {percentage:.1f}%")`

➡️ Tomorrow

Lists and dictionaries: one variable is great for one value — but data usually comes in collections, like a week of temperatures or all the details about one student. Tomorrow you'll learn how to store many values together.

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. Your profile

```
name = "Sara"
age = 21
height = 1.65
```

2. Game score

```
score = 10

# Add 5 points
score += 5

# Double the score
score *= 2
```

3. Numbers that arrived as text

```
price_text = "19.99"
quantity_text = "3"

# Convert and calculate total
total = float(price_text) * int(quantity_text)
```

4. Average temperature

```
monday = 17.2
tuesday = 21.9
wednesday = 19.4

average_temp = (monday + tuesday + wednesday) / 3
print(f"Average: {average_temp:.1f} degrees")
```

5. Challenge: test report

```
student = "Lina"
correct = 17
total_questions = 20

percentage = correct / total_questions * 100
print(f"{student} scored {percentage:.1f}%")
```